

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

«Алгоритмы и структуры данных»

отчет по блоку задач №3 (Яндекс.Контест)

Выполнил:

Студент группы Р3210

Давидюк Антон

Задача №1 I. Машины

Разбор: `nxt` хранит для каждого шага индекс следующего появления той же машинки, `INF` используется как маркер, что машинка больше не понадобится. `last` запоминает последнюю встреченную позицию каждой машинки при обратном проходе по массиву запросов. `cache` хранит активные машинки на полу в виде пар {индекс следующего запроса, `id` машинки}, множество поддерживает порядок по возрастанию индекса, поэтому последний элемент (`prev(cache.end())`) это машинка, которая понадобится позже всех. `cur_next` и `present` обеспечивают доступ за $O(1)$ к использованию и статусу на полу. В основном цикле алгоритм проверяет наличие машинки: если её нет, счётчик операций `ops` увеличивается. При заполнении пола (`cache.size() == K`) извлекается последний элемент из `cache`, его статусы обнуляются, место освобождается. Новая машинка добавляется в `cache`. Если машинка уже на полу, её запись в `cache` обновляется на `next_use`. Отдельная ветка $K \geq N$ мгновенно считает количество уникальных машинок, избегая лишних вычислений с контейнерами.

Алгоритмическая сложность:

1. По времени: Предварительный расчёт следующих вхождений выполняется за $O(P)$. Основной цикл делает P итераций, внутри каждой операции с `set` занимают $O(\log K)$. Итоговая сложность $O(P \log K)$.
2. По памяти: Векторы занимают $O(P + N)$ памяти. Множество `cache` хранит не более K элементов. Вспомогательные структуры используют линейное количество памяти относительно входных данных. Итоговая сложность $O(P + N)$.

Код:

```
#include <iostream>
#include <set>
#include <utility>
#include <vector>

std::vector<int> NextIdx(const std::vector<int>& req, int cars_cnt) {
    int P = req.size(), INF = P + 1;
    std::vector<int> nxt(P, INF), last(cars_cnt + 1, INF);
    for (int i = P - 1; i >= 0; --i)
        nxt[i] = std::exchange(last[req[i]], i);
    return nxt;
}

long long solve(int total_cars, int K, const std::vector<int>& req, const std::vector<int>& nxt) {
    std::set<std::pair<int, int>> cache;
    std::vector<int> cur_next(total_cars + 1, -1);
    std::vector<bool> present(total_cars + 1, false);
    long long ops = 0;

    for (int i = 0, P = req.size(); i < P; ++i) {
        int car = req[i], nxt_use = nxt[i];
        if (present[car]) {
            cache.erase({ cur_next[car], car });
            cur_next[car] = nxt_use;
            cache.insert({ nxt_use, car });
        }
        else {
            ++ops;
            if ((int)cache.size() == K) {
                auto it = std::prev(cache.end());
                present[it->second] = false;
                cur_next[it->second] = -1;
                cache.erase(it);
            }
        }
    }
}
```

```

        present[car] = true;
        cur_next[car] = nxt_use;
        cache.insert({ nxt_use, car });
    }
}
return ops;
}

int main() {
    int N, K, P;
    if (!(std::cin >> N >> K >> P))
        return 0;

    std::vector<int> req(P);
    for (int& x : req)
        std::cin >> x;

    if (K >= N) {
        std::vector<bool> seen(N + 1, false);
        long long ans = 0;
        for (int c : req)
            if (!seen[c]) {
                ++ans;
                seen[c] = true;
            }
        std::cout << ans << std::endl;
        return 0;
    }

    auto nxt = NextIdx(req, N);
    std::cout << solve(N, K, req, nxt) << std::endl;
}

```

Задача №2 J. Гоблины и очереди

Разбор:

Используется структура из двух deque: frontSegment и backSegment, которые моделируют очередь с вставкой в середину. Размер frontSegment всегда либо равен размеру backSegment, либо превышает его на единицу. Это гарант, того что граница между сегментами всегда соответствует середине общей очереди, а при нечётной длине центральный элемент хранится в конце frontSegment. При запросе + (обычный гоблин) элемент добавляется в конец backSegment, после чего, если баланс нарушен, первый элемент backSegment перемещается в конец frontSegment. При запросе * (привилегированный гоблин) элемент добавляется в конец frontSegment (позиция сразу за центром), при необходимости последний элемент frontSegment перебрасывается в начало backSegment. При запросе - элемент извлекается из начала frontSegment (или из backSegment, если первый пуст), после чего также выполняется проверка и восстановление баланса.

Алгоритмическая сложность:

1. По времени: Все операции с deque выполняются за $O(1)$. восстановление баланса на каждом запросе перемещает не более одного элемента, что занимает $O(1)$. Поскольку всего обрабатывается N запросов, временная сложность алгоритма составляет $O(N)$
2. По памяти: контейнеры deque в сумме хранят не более N целых чисел (по одному на каждого гоблина, находящегося в очереди в моменте). Вспомогательные переменные занимают константный объём памяти. сложность по памяти $O(N)$

Код:

```
#include <deque>
#include <iostream>

int main() {
    int queryCount;
    if (!(std::cin >> queryCount))
        return 0;

    std::deque<int> frontSegment;
    std::deque<int> backSegment;

    char operation;
    int goblinId;

    for (int queryIdx = 0; queryIdx < queryCount; ++queryIdx) {
        std::cin >> operation;

        if (operation == '+') {
            std::cin >> goblinId;
            backSegment.push_back(goblinId);

            if (backSegment.size() > frontSegment.size()) {
                frontSegment.push_back(backSegment.front());
                backSegment.pop_front();
            }
        }
        else if (operation == '*') {
            std::cin >> goblinId;
            frontSegment.push_back(goblinId);
            if (frontSegment.size() > backSegment.size() + 1) {
                backSegment.push_front(frontSegment.back());
                frontSegment.pop_back();
            }
        }
        else if (operation == '-') {
            int removedGoblin;
            if (!frontSegment.empty()) {
                removedGoblin = frontSegment.front();
                frontSegment.pop_front();
            }
            else {
                removedGoblin = backSegment.front();
                backSegment.pop_front();
            }

            if (backSegment.size() > frontSegment.size()) {
                frontSegment.push_back(backSegment.front());
                backSegment.pop_front();
            }

            std::cout << removedGoblin << std::endl;
        }
    }

    return 0;
}
```

Задача №3 К. Менеджер памяти-1

Разбор:

Используется двойная индексация свободных блоков памяти, это позволяет одновременно искать подходящий участок и объединять соседние области при освобождении. Контейнер `by_addr` хранит соответствие начального адреса к длине блока, это даёт возможность находить левого и правого соседей через итераторы `prev/next`. Контейнер `by_size` хранит пары {длина, начальный адрес}, отсортированные по длине, благодаря чему поиск первого блока, достаточного для размещения, выполняется за $O(\log M)$. Лямбды `add` и `rem` обеспечивают синхронизацию обоих контейнеров. При запросе на выделение алгоритм находит минимальный подходящий блок, при необходимости делит его и возвращает остаток в пул свободных. При освобождении блок возвращается в пул, после чего проверяется смежность с левым и правым соседями по адресам и выполняется слияние. `hist` фиксирует историю успешных выделений, это позволяет обрабатывать запросы на освобождение и игнорировать уже выполненные или отклонённые.

Алгоритмическая сложность:

1. По времени: Каждый запрос выполняется за $O(\log M)$, где M количество запросов. Все ключевые операции имеют сложность $O(\log K)$, где K текущее количество свободных блоков ($K \leq M$). Слияние соседних участков требует константного числа таких операций. Сложность по времени $O(M \log M)$.
2. По памяти: В структурах данных одновременно хранится не более M свободных блоков и активных выделений. Каждый из трёх основных контейнеров (`by_addr`, `by_size`, `hist`) потребляет линейную память относительно количества запросов $O(M)$. Вспомогательные переменные и итераторы используют константный объём. Сложность по памяти $O(M)$.

Код:

```
#include <iostream>
#include <map>
#include <set>

int main() {
    int n, m;
    std::cin >> n >> m;

    std::map<int, int> by_addr;
    std::set<std::pair<int, int>> by_size;

    auto add = [&](int s, int l) { by_addr[s] = l; by_size.insert({ l, s }); };
    auto rem = [&](int s) { int l = by_addr[s]; by_addr.erase(s); by_size.erase({ l, s }); };

    add(1, n);

    std::map<int, std::pair<int, int>> hist;

    for (int qid = 1, req; qid <= m; ++qid) {
        std::cin >> req;
        if (req > 0) {
            auto it = by_size.lower_bound({ req, 0 });
            if (it == by_size.end()) {
                std::cout << "-1\n";
            }
            else {
                int start = it->second, len = it->first;
                rem(start);
                if (len > req) add(start + req, len - req);
                std::cout << start << '\n';
            }
        }
    }
}
```

```

        hist[qid] = { start, req };
    }
}
else {
    int target = -req;
    auto h = hist.find(target);
    if (h == hist.end()) continue;
    int start = h->second.first, len = h->second.second;
    hist.erase(h);
    add(start, len);

    auto cur = by_addr.find(start);
    if (cur != by_addr.begin()) {
        auto L = prev(cur);
        if (L->first + L->second == start) {
            int ns = L->first, nl = L->second + len;
            rem(L->first); rem(start); add(ns, nl);
            start = ns; len = nl; cur = by_addr.find(start);
        }
    }
    auto R = next(cur);
    if (R != by_addr.end() && R->first == start + len) {
        int nl = len + R->second;
        rem(start); rem(R->first); add(start, nl);
    }
}
}
return 0;
}

```

Задача №4 L. Минимум на отрезке

Разбор: Очередь хранит индексы элементов последовательности, поддерживается строгое условие: значения, соответствующие индексам в очереди, всегда идут в неубывающем порядке. Благодаря этому элемент, расположенный в позиции `head`, всегда минимальный для текущего положения скользящего окна. На каждом шаге итерации алгоритм выполняет три ключевые операции: 1) из хвоста последовательно удаляются индексы, значения которых больше или равны текущему элементу, так как они больше не могут претендовать на роль 2) индекс текущего элемента добавляется в хвост очереди. 3) проверяется, не вышел ли индекс из `head` за левую границу окна, и при необходимости `head` сдвигается вправо. Как только окно заполняется, минимумом становится значение по индексу `monotonic_queue[head]`, которое выводится.

Алгоритмическая сложность:

1. По времени: Каждый индекс добавляется в очередь ровно один раз и удаляется из неё не более одного раза (либо при поддержании неубывающего порядка из хвоста, либо при выходе за границы окна из головы). Это обеспечивает сложность $O(1)$ на обработку каждого элемента. В сложность алгоритма $O(N)$.
2. По памяти: массив `data` и вектор `monotonic_queue` хранят не более N целых чисел каждый. Указатели `head`, `tail` и вспомогательные переменные занимают константный объём памяти. Итоговая сложность по памяти составляет $O(N)$.

Код:

```
#include <iostream>
#include <vector>

int main() {

    int sequence_len, wind_size;
    if (!(std::cin >> sequence_len >> wind_size)) {
        return 0;
    }

    std::vector<int> data(sequence_len);
    for (int indx = 0; indx < sequence_len; ++indx) {
        std::cin >> data[indx];
    }

    std::vector<int> monotonic_queue(sequence_len);
    int head = 0;
    int tail = 0;

    for (int current_indx = 0; current_indx < sequence_len; ++current_indx) {
        int current_val = data[current_indx];

        while (head < tail && data[monotonic_queue[tail - 1]] >= current_val) {
            --tail;
        }

        monotonic_queue[tail++] = current_indx;

        if (monotonic_queue[head] <= current_indx - wind_size) {
            ++head;
        }

        if (current_indx >= wind_size - 1) {
            std::cout << data[monotonic_queue[head]];
            if (current_indx < sequence_len - 1) {
                std::cout << ' ';
            }
        }
    }
    std::cout << std::endl;

    return 0;
}
```